# CPSC 220 Projects

version 1.0

*Last generated: August 28, 2016*

© 2016 University of Mary Washington. All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

# Table of Contents

# Shared Secret

**Learning Objective(s):**

- Understand modulo arithmetic.
- Understand the basics of public-private keys.

## Shared Secret Project

Note: This project was created based upon Public Key Cryptography chapter of *9 Algorithms that Changed the Future* by John MacCormick. John introduces the public-private technique with paint mixing. You can see a similar treatment of paint mixing at Diffie-Hellman Key Exchange (https://en.wikipedia.org/wiki/Diffie–Hellman_key_exchange).

The Shared Secret is your first project. We know how to develop a basic Java program using `main`, `System.out` for output, a `Scanner` for input, methods, and expressions. Those are the only programming features required for this project.

1. Develop a SharedSecret class that satisfies the following.
    - a. The SharedSecret class shall be in a SharedSecret.java file that is submitted on Canvas.
    - b. The SharedSecret class shall contain code that satisfies the requirements as specified in this project.
    - c. The SharedSecret class shall adhere to the basic design as provided in this project.
    - d. Creating the SharedSecret.java file is an individual assignment. You shall not use your team members.
2. Create a `main` method in SharedSecret that tests your implementation. This specification provides a sample run of `main`.
    - a. This is a team component. Team members shall work together to create the same SharedSecret.java file.
3. Establish a SharedSecret with a team member.
    - a. Team members shall create public, public-private, and private numbers. Team members shall use these numbers as described in this project to create a shared secret number.

- b. Teams shall decide how to satisfy this requirement. For examples,
    - i. Team members could email each other their public-private numbers and then email their resulting shared secret numbers.
    - ii. Teams can meet and exchange their public-private numbers and discuss their shared secret numbers.
- c. This is a team component. Your UMW Honor Code pledge on your Canvas submission indicates that your team has completed this component of the project.

## Modulo and Clock Arithmetic

Clock arithmetic uses arithmetic operators such that the resulting answer remains in a range. Our 12-hour clock provides a simple example of clock arithmetic. Our 12-hour clock has the numbers 1, 2, 3, … 12. If it is 9:00 AM, and your friend says meet me at the gym in 6 hours, you know to meet at 3:00 PM. In normal arithmetic 9 + 6 is 15, but in clock arithmetic the answer has to be between 1 and 12. In this example, you subtract 12 from 15 to get 3.

The modulo operator creates clock arithmetic answers. Consider the following example using Modulo 11 in the BlueJ Codepad. Modulo 11 works with 11 numbers that are 0, 1, 2, …, 10. Pay attention to the use of `Math.pow` in the following examples because we will use it in Shared Secret. Since we want the answer to be an `int``, the examples show two calls to` Math.pow `. The first results in a` double `. The secon`

```
> 5 % 11
5   (int)
> 12 % 11
1   (int)
> (12 + 5) % 11
6   (int)
> (6 * 7) % 11
9   (int)
> Math.pow(2,5) % 11
10.0   (double)
> (int)(Math.pow(2,5)) % 11
10   (int)
> 11 % 11
0   (int)
> (11 * 11) % 11
0   (int)
```

# Numbers - Public, Private, Public-Private, and Shared-Secret

The code for this project is not difficult, but it mimicks public-private keys, which are important on the internet. The goal is to use public numbers and public-private numbers from two teammates to create a shared secret number that is the same for both teammates, which they can use to encrypt information that is exchanged between them.

## Public Numbers

The public numbers are `moduloNumber` and a `base`. For this project, we select 11 as our modulo and 2 as our base; however, our algorithms shall be designed such that these public numbers can be changed. All team members shall know the public numbers.

## Private Numbers

Each team member shall select a `privateNumber` that is between 0 and 10 inclusive. Private numbers shall not be shared with team members.

## Public-private Numbers

Each team member shall compute the public-private number in accordance with the following expression.

$(\texttt{base}^{\texttt{privateNumber}}) \texttt{ \% moduloNumber}$

## Shared-secret Numbers

Two team members create a shared-secret number by using their private number and the public-private number of their team mate in accordance with the following expression.

$(\texttt{teamMatePublicPrivateNumber}^{\texttt{myPrivateNumber}}) \texttt{ \% moduloNumber}$

Any pair of teammates will share the same secret number, which could be used for encrypting information that is exchanged between them.

## Example

This example uses the following.

- `moduloNumber` is 11

- `base` is 2
- Gusty selects 5 as his private number. Gusty's public-private number can be seen by all team members and it is

```
> (int)(Math.pow(2,5)) % 11
10   (int)
```

- Coletta selects 2 as her private number. Coletta's public-private number can be seen by all team members and it is

```
> (int)(Math.pow(2,2)) % 11
4   (int)
```

- Emily selects 7 as her private number. Emily's public-private number can be seen by all team members and it is

```
> (int)(Math.pow(2,7)) % 11
7   (int)
```

- Gusty and Coletta's shared secret number is 1.
    - Gusty uses his private number (5) and Coletta's public-private number (4) to computes his shared secret number with Coletta as follows.

    ```
    > (int)(Math.pow(4,5)) % 11
    1   (int)
    ```

    - Coletta uses her private number (2) and Gusty's public-private number (10) to compute her shared secret number with Gusty as follows.

    ```
    > (int)(Math.pow(10,2)) % 11
    1   (int)
    ```

- Gusty and Emily's shared secret number is 10.

- Gusty uses his private number (5) and Emily's public-private number (7) to computes his shared secret number with Emily as follows.

```
> (int)(Math.pow(7,5)) % 11
10   (int)
```

- Emily uses her private number (7) and Gusty's public-private number (10) to compute her shared secret number with Gusty as follows.

```
> (int)(Math.pow(10,7)) % 11
10   (int)
```

- Coletta and Emily's shared secret number is 5.
  - Coletta uses her private number (2) and Emily's public-private number (7) to computes his shared secret number with Emily as follows.

```
> (int)(Math.pow(7,2)) % 11
5   (int)
```

  - Emily uses her private number (7) and Coletta's public-private number (4) to compute her shared secret number with Gusty as follows.

```
> (int)(Math.pow(4,7)) % 11
5   (int)
```

## SharedSecret Class Organization

Your `SharedSecret` class shall consist of the following methods.

- `public static int publicPrivate(int base, int moduloNumber, int privateNumber)` - computes a public-private number using the expression described above.

- `public static int sharedSecret(int teamMatePublicPrivateNumber, int myPrivateNumb` - computes a shared-screte number using the expression described above.
- `public static void main(String[] args)` - uses `System.out`, a `Scanner` object, and calls `publicPrivate` and `sharedSecret` to accmplish the following, which shows three calling of `main` from BlueJ.

```
Shared Secret Project
Enter base: 2
Enter moduloNumber: 11
Enter Name 1: Gusty
Enter private number for Gusty: 5
Enter Name 2: Coletta
Enter private number for Coletta: 2
Gusty Private: 5 PublicPrivate: 10 SharedSecret: 1
Coletta Private: 2 PublicPrivate: 4 SharedSecret: 1

Shared Secret Project
Enter base: 2
Enter moduloNumber: 11
Enter Name 1: Gusty
Enter private number for Gusty: 5
Enter Name 2: Emily
Enter private number for Emily: 7
Gusty Private: 5 PublicPrivate: 10 SharedSecret: 10
Emily Private: 7 PublicPrivate: 7 SharedSecret: 10

Shared Secret Project
Enter base: 2
Enter moduloNumber: 11
Enter Name 1: Coletta
Enter private number for Coletta: 2
Enter Name 2: Emily
Enter private number for Emily: 7
Coletta Private: 2 PublicPrivate: 4 SharedSecret: 5
Emily Private: 7 PublicPrivate: 7 SharedSecret: 5
```

## Modulo Number and Key

The modulo number determines number of shared secret numbers, and is referred to as the key. With a modulo of 11, we have 11 shared secret numbers, which is small. Even though our algorithm allows two people to create a shared secret number from public and public-private information, it would not take many

guesses to determine that shared secret number of any pair of individuals. What is needed is so many shared secret numbers that it becomes statistically difficult to guess. We can represent 11 numbers with 4 bits ($2^4$ is 16) or a 4 bit key. We can represent 256 numbers with 8 bits, 65536 numbers with 16 bits, and over 4 billion numbers with a 32 bit key. Real uses of these algorithms use keys that are 128 bits, which produces over a trillion trillion numbers.

## Using Exponentiation

You want to select an arithmetic operation to compute public-private and shared-secret numbers that is hard to reverse. For example, you should have a difficult time to translate someone's public-private number back to their private number using the public numbers. Multiplication is easy to reverse. Exponentiation is hard to reverse.

## Integer Overflow in Our Implementation

We implemented our methods with parameters that allow us to change the base and modulo, but exponentiation can easily overflow an `int`, especially considering we casted the return value of `Math.pow` to an `int` prior to performing the modulo. Suppose I use a base of 2 and a modulo of 17, which allows us to choose private numbers between 0 and 16.

- Gusty - Private: 5 PublicPrivate 15
- Coletta - Private: 13, PublicPrivate: 15
- Gusty and Coletta's SharedSecret number is 2; however, our implementation overflows an int creating the wrong SharedSecret number for Coletta.
    - Gusty-Coletta SharedSecret: (int)($15^5$) % 17
        - $15^5$ is 759375.0, which fits into an `int`.
    - Coletta-Gusty SharedSecret: (int)($15^{13}$) % 17
        - $15^{13}$ is 1.946195068359375E15, which does not fit into an `int`. The following shows the result of this casting `Math.pow(15,13)` to an `int` prior to performing `%17`, and casting `(Math.pow(15,13)%17)` to an `int`.

```
> Math.pow(15,13)
1.946195068359375E15   (double)
> (int)Math.pow(15,13)
2147483647   (int)
> Integer.MAX_VALUE
2147483647   (int)
> (int)Math.pow(15,13) % 17
8   (int)
> (int)(Math.pow(15,13) % 17)
2   (int)
```

# Cipher

**Learning Objective(s):**

- Understand elementary encryption algorithms
- Think about encryption as it relates to events in life such as the Internet.
- Understand ASCII and UTF encodings of char.
- Understand the char type can be used in arithmetic expressions.
- Practice writing Java classes and creating objects.
- Practice writing a Java tester class.
- Practice writing JavaDoc.
- Begin to think about Project 5 - Your Choice.

## Cipher Project

The Cipher Project continues your journey on learning to write Java programs. For this project you will use control flow in implementing your classes. You will continue exploring Java Strings. You will use Java `char` types and learn that you can apply arithmetic operations on chars. For the Cipher project you will complete the following.

1. Develop a Cipher class that satisfies the following.
    - a. The Cipher class shall be in a Cipher.java file that is submitted on Canvas.
    - b. The Cipher class shall contain code for a Caesar cipher and an Augustus cipher.
    - c. The Caesar cipher requirements, design, discussion, and code are provided in this paper. The code to implement the Caesar cipher is provided in this specifiction. You begin with code and add your implementation of the Augustus cipher.
    - d. The Augustus cipher requirements and discussion are provided in this paper. You shall design the Augustus cipher algorithm and implement the code. The Augustus cipher code shall be placed in your Cipher.java file.
    - e. You shall create JavaDoc that properly documents your Cipher class. The JavaDoc shall be placed in your Cipher.java file.

    - f. Creating the Cipher.java file is an individual assignment. You shall not use your team members.

2. Develop a Cipher Tester class that tests both the Caesar and Augustus ciphers that are in your Cipher.java file.
    - a. The Cipher Tester class shall be in a CipherTester.java file that submitted on Canvas.
    - b. This is a team component. Team members shall work together to create the same CipherTester.java file.
3. Decode messages that I email to you.
    - a. I will email to you several encrypted messages and their corresponding keys. You will reply to the email with the decrypted versions of these messages.
    - b. This is an individual component. You shall use your Cipher class created in number 1 to perform the decrypt.
4. Exchange messages and keys.
    - a. Team members shall create several messages, encrypt them, and provide the encrypted messages and corresponding keys to other team members. The recipients of the encrypted messages shall decrypt them and inform the sender(s) what the decrypted message(s) is.
    - b. Teams shall decide how to satisfy this requirement. For examples,
        - i. One member could email the other members encrypted messages and keys.
        - ii. Pairs of members could exchange encrypted messages and keys.
    - c. This is a team component. Your UMW Honor Code pledge on your Canvas submission indicates that your team has completed this component of the project.
5. Provide a list of ideas for your choice project in a file ProjectChoice.docx. The .docx extension is an example. You can submit a PDF or text file or MacOS Pages file.
    - a. The goal of this is for you to begin thinking about your choice project. You can change your mind as the class continues.
    - b. This is an individual component; however, you can discuss ideas with anyone.

6. Extra Credit: In order to receive extra credit, you must negotiate with me what you plan to do prior to submitting your project. We will agree on your task and how many points you will be awarded.
    - a. How do you know that your credit card information is not compromised when purchasing on the Internet?
    - b. New encryption apps allow people to exchange encrypted messages. The US Government wants a backdoor to view encrypted messages of suspected criminals. Discuss whether or not you think the US Government should have this backdoor to bypass the encryption.
    - c. Select a modern encryption technique and describe how it works.
    - d. How does the encryption algorithm of WWII German messages compare to modern encryption algorithms? The movie, Immitation Game shows Alan Turing leading a team that broke the German encryption by creating the Enigma "Computer".
    - e. Determine a way to include the SecretKey as part of this cipher.

## Cipher Class Interface Requirements

The high-level requirements for the Cipher class are specified by describing the public interface, which are the public methods of the class. You must conform to this interface because I will test your program using my tester. If you misspell your caesarEncrypt method as ceasarEncrypt, your Cipher class will not work with my tester because my tester will get an undefined method compile error.

- `public Cipher(int secretKey)` – constructor for your Cipher class. The secretKey is used by encrypt and decrypt methods of the Cipher class.
- `public String caesarEncrypt(String s)` – returns a `String` that is the Caesar encrypted version of the `String s`. This method shall use the Caesar code that is provided in this specification. The Caesar algorithm requirements are discussed and implemented in this specification.
- `public String caesarDecrypt(String s)` – returns a `String` that is the Caesar decrypted version of the `String s`. This method shall use the Caesar code this is provided in this specification. The Caesar algorithm requirements are discussed and implemented in this specification.

- `public String augustusEncrypt(String s)` – returns a `String` that is the Augustus encrypted version of the `String s`. This method shall implement the Augustus algorithm requirements in this specification.
- `public String augustusDecrypt(String s)` – returns a `String` that is the Augustus decrypted version of the `String s`. This method shall implement the Augustus algorithm requirements in this specification.

# Caesar's Cipher

## Introduction Caesar's Cipher

Caesar's cipher encrypts (i.e., scrambles in a reversible way) messages by "rotating" each letter by k positions, wrapping around from 'Z' to 'A' as needed. The following Wikipedia article may be read: Caesar Cipher (http://en.wikipedia.org/wiki/Caesar_cipher)

## Caesar Cipher Algorithm Requirements

Let p be an unencrypted, plaintext message (e.g., Hello Gusty). If pi is the ith character in p and k is a non-negative integer called the secret key, then each letter, ci, in the cipher text, c, is computed as:

```
ci = (pi + k) % 26
```

This formula is a precise and concise way to express the Caesar cipher algorithm. You will discover that Caesar's cipher is not the most secure cipher, but fun to implement! A few examples will help to understand the formula.

### Example 1

Let the secret key be 1, which means the cipher text is shifted 1 character from the plain text. The following shows upper case letters shifted 1 character.

```
Plain  A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Cipher B C D E F G H I J K L M N O P Q R S T U V W X Y Z A
```

Using the secret key of 1 to encrypt the plaintext message Dog results in the cipher text message Eph because 'D' is encrypted as E, 'o' is encrypted as 'p' and 'g' is encrypted as 'h'. Encrypting Zac results in Abd.

Example 2

Let the secret key be 13. The letter A shifted 13 results in N. The following shows upper case letters shifted 13 character.

```
Plain  A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Cipher N O P Q R S T U V W X Y Z A B C D E F G H I J K L M
```

Using the secret key of 13 to encrypt the plaintext message Be sure to drink your Ovaltine! results in the cipher text message Or fher gb qevax lbhe Binygvar!

```
Plain  Be sure to drink your Ovaltine!
Cipher Or fher gb qevax lbhe Binygvar!
```

## Caesar Cipher Additional Requirements

The Caesar cipher algorithm shall only rotate upper and lower case characters. An upper case character shall be rotated to an upper case character and a lower case character shall be rotated to a lower case character. Spaces and punctuation characters shall be the same in both the plain text message and the cipher message. For example, the string Zac's FAV # is 123! encrypted with a key of 1 is Abd't GBW # jt 123!

Although there exist only 26 letters in the English alphabet, you may not assume key passed to the Cipher constructor will be less than or equal to 26. Your program shall work for all non-negative values of key. If the key is greater than 26 you shall still have alphabetical characters in the output. For example, if the key is 27. 'A' shall not become '[' even though '[' is 27 positions away from 'A'. Instead 'A' shall become 'B' since 27 modulo 26 is 1.

The following shows the Cipher class being used with the Caesar cipher methods.

```
Cipher cipher = new Cipher(27);
String s = cipher.caesarEncrypt("Dog") // s is Eph
String s = cipher.caesarDecrypt(cipher.encrypt("Dog")) // s is
Dog
Cipher salad = new Cipher(13);
String u = salad.caesarEncrypt("Ovaltine!") // u is Binygvar!
```

# Java char Can Be Used in Arithmetic Expressions

Lets talk about manipulating Java `char` s so they remain uppercase or lowercase. Lowercase `char` letters ‘a’ through ‘z’ are represented as the numbers 97 through 122. Uppercase `char` letters ‘A’ through ‘Z’ are represented as the numbers 65 through 90. If Z is rotated by 4, it becomes D. In terms of numbers, 90 becomes 68. Java allows you to use `char` variables and literals with arithmetic operators. For example,

```
char c = (char)('A' + 4);
```

assigns the `char ‘E’` to the variable `c` . In rotating characters, you need an algorithm that adds the key to an uppercase letter and keeps the resulting value in the range of uppercase letters. Suppose the key is a 2, the following is a bad algorithm because it rotates Z outside the letter range.

```
char c = (char)('Z' + 2); // c will be 92, which is the backsla
sh \
```

Consider the following expression.

- `90 - 65` , which represents `'Z' - 'A'` .

This expression computes the offset from A to Z, which is 25.

When we add our key of 2 to 25 to get 27. If we perform modulo (% operator) 26 on 27, we get 1, which when added to 65 gives us 66, which is B. The following two expressions capture this discussion. The first expression uses numbers, and the second expression uses Java `char` . The two expressions are equivalent.

- `65 + ( (90 - 65) % 26)` , which yields 66 or the number for B.
- `'A'+ ( ('Z' - 'A') % 26)` , which yields 66 or the number for B.

You have to cast an integer expression with `(char)` when assigning it to a `char` variable. The following expression properly rotates an uppercase `char` by key such that it rotates into an uppercase `char` .

```
char c = 'X';
c = (char)('A' + (c - 'A' + key) % 26);
```

You will need a similar expression for rotating lowercase characters.

## Java Strings and chars

The previous section provided techniques for rotating Java `char` s; however, the Cipher class encrypt and decrypt methods manipulate Java Strings, and `String` variables are immutable. Java has several classes for strings. A `StringBuffer` class is similar to `String` - they both have a sequence of characters, but `StringBuffer` variables are mutable with methods that allow you to change them. The following code snippet rotates the first character of Coffee by 4.

```
class ExampleCode {
    private static String replaceCharAt(String s, int i, char
c){
        StringBuffer buf = new StringBuffer(s);
        buf.setCharAt(i, c);
        return buf.toString();
    }

    public static void main(String[] args) {
       String word = "Coffee";
       int key = 4; // rotate by 4
       char c = word.charAt(0); // get char from word
       if (Character.isLetter(c)) {
          if (Character.isUpperCase(c)) {
             c = (char)('A' + (c - 'A' + key) % 26);
          } else {
             c = ...; // code missing
          }
       }
       word = replaceCharAt(word, 0, c);
       System.out.println(word);
    }

}
```

The above code uses a `StringBuffer` (https://docs.oracle.com/javase/8/docs/api/java/lang/StringBuffer.html). You can also use a `StringBuilder` (https://docs.oracle.com/javase/tutorial/java/data/buffers.html). Both `StringBuffer` and `StringBuilder` classes are sequences of characters and

they allow you to modify them. A `StringBuffer` is thread-safe, which is something we do not study in this course. Threads are multiple processes running concurrently. Thread-safe means the multiple processes cannot mess-up each other accessing the ```StringBuffer.

## Decrypting

If we rotate Dog by 1 we get Eph. What do we have to rotate Eph by in order to get Dog? Can you use caesarEncrypt() with a general form of the initial key to implement caesarDecrypt()? You should be able to answer these questions by studying the code in the next section.

## Cipher Class with Caesar Code

The following code is the Cipher.java file with Caesar cipher code. You shall use this as the starting point for you Cipher.java file. You shall add your implementation of the Augustus cipher. You shall place JavaDoc in your Cipher.java for both the Caesar cipher and the Augustus cipher.

```
public class Cipher {
    int key;
    // You may need additional instance variables

    public Cipher(int secretKey) {
        key = secretKey % 26;
        // You may need additional code
    }

    private String replaceCharAt(String s, int i, char c) {
        StringBuffer buf = new StringBuffer(s);
        buf.setCharAt(i, c);
        return buf.toString();
    }

    private String encryptLocal(String s, int key) {
     // for (char c : s.toCharArray()) {  // alternative for ea
ch loop
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (Character.isLetter(c)) {
                if (Character.isUpperCase(c)) {
                    c = (char)('A' + (c - 'A' + key) % 26);
                } else {
                    c = (char)('a' + (c - 'a' + key) % 26);
                }
                s = replaceCharAt(s, i, c);
            }
        }
        return s;
    }

    public String caesarEncrypt(String s) {
        return encryptLocal(s, key);
    }

    public String caesarDecrypt(String s) {
        return encryptLocal(s, 26-key);
    }

    public String augustusEncrypt(String s) {
        // add code
    }

    public String augustusDecrypt(String s) {
        // add code
```

```
    }
}
```

# Augustus Cipher

## Introduction to Augustus Cipher

When Caesar realized that his enemies were easily cracking his Caesar Cipher, he consulted with his cabinet to create the Augustus Cipher. The Augustus Cipher is more secure than the Caesar Cipher, but it is not as secure as today's modern encryption algorithms. Nor is it as secure as the Enigma as shown in the Imitation Game.

## Augustus Cipher Algorithm Requirements

The Augustus Cipher key is a number that is applied in two steps.

1. The first step uses the key in the Caesar Cipher described previously. For example the first step with a key of 13 encrypts Hello as Uryyb.
2. The second step uses each individual digit of the key as a Caesar Cipher that is applied to the encrypted text of the first step.

Let p be the plain text string, let k be the key, and let cc be the Caesar cipher text created from p using k. Let cci be the ith character in cc and ki be the ith number in the key (i.e., if k is 479, then k2 is 9). Each letter, ci, in the cipher text, c, is computed as:

```
ci = (cci + kj) %  26
```

When the length of the key is shorter than the length of the text, you cycle back to the first digit of the key. The examples will demonstrate this.

### Example 1

This example uses the Augustus Cipher to encrypt Hello! with a key of 13. The following shows step 1 of the Augustus Cipher.

```
Plain  A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Cipher N O P Q R S T U V W X Y Z A B C D E F G H I J K L M

Plain Text  Hello!
1st Cipher   Uryyb!
```

The following shows step 2 of the Augustus Cipher where the individual digits of the key are used as Caesar Ciphers on individual letters of cipher text from step 1.

- U rotated by 1 to get V
- r rotated by 3 to get u
- y rotated by 1 to get z
- y rotated by 3 to get b
- b rotated by 1 to get c
- ! is not rotated

The following is a tabular form step 2 of the Augustus Cipher.

```
  U  r  y  y  b  !
+ 1  3  1  3  1
------------------
  V  u  z  b  c  !
```

### Example 2

This example uses the Augustus Cipher to encrypt Hello! with a key of 72345. You will notice that 72345 % 26 is 13 so the first step of the Augustus Cipher in this example is the same as the previous example.

```
Plain Text  Hello!
1st Cipher  Uryyb!
```

The second step uses 7, 2, 3, 4, 5 as follows.

```
  U  r  y  y  b  !
+ 7  2  3  4  5
------------------
  B  t  b  c  g  !
```

### Example 3

This example uses the Augustus Cipher to encrypt Hello! with a key of 715. You will notice that 715 % 26 is 13 so the first step of the Augustus Cipher in this example is the same as Example 1.

```
Plain Text  Hello!
1st Cipher  Uryyb!
```

The second step uses 7, 1, 5 as follows.

```
  U  r  y  y  b  !
+ 7  1  5  7  1
-----------------
  B  s  d  f  c  !
```

## Augustus Cipher Additional Requirements

The Augustus cipher has the same additional requirements as the Caesar cipher. You shall only rotate upper and lower case characters. An upper case character shall be rotated to an upper case character and a lower case character shall be rotated to a lower case character. Spaces and punctuation characters shall be the same in both the plain text message and the cipher message. Your program shall work for all non-negative values of key. If the key is greater than 26 you shall still have alphabetical characters in the output.

The following shows the Cipher class being used with the Augustus cipher methods.

```
Cipher cipher = new Cipher(13);
String s = cipher.augustusEncrypt("Hello!"); // s is Vuzbc!
Cipher salad = new Cipher(72345);
String u = salad.augustusEncrypt("Hello!"); // u is Btbcg!
```

## Cipher Tester

Each team shall create one CipherTester class that all team members use to test their individual Cipher classes. A team should think of test cases that fully test the requirements of the Cipher class – both Caesar and Augustus. Everyone should realize that the Cipher class is an individual component. The team's CipherTester may contain test cases that fail on an individual's Cipher class. The team may NOT help an individual debug the individual's Cipher class. Individuals are responsible for debugging their own Cipher class.

The following suggestions are not a thorough collection of tests, but should serve as a starting point for you to think of your own set of test cases.

- Strings that contain upper and lower case characters
- Strings that contain spaces and punctuation.
- Simple keys such as 1 and 2.
- Larger keys such as 46 and 1234532.
- Strings and keys that cause letters to rotate from the end of the alphabet to the front of the alphabet.
- Decrypting an encrypted string should produce the original string.
    - `cipher.augustusDecrypt(cipher.augustusEncrypt("Gusty"))` should return "Gusty"

The CipherTester class shall print expected values and actual values.

# Endor Smints

**Learning Objective(s):**

- Practice writing a simulation game.
- Understand how to maintain state within a class.
- Understand how to incrementally develop a program.
- Understand how to develop test cases prior to developiong code.
- Have fun creating the EndorSmints game.

## Endor Smĭnts – Description and Requirements

The Star Wars gang is flying in the Millennium Falcon toward the Hapsacean Galaxy, many light years away. The hyperdrive engine, required for such a long journey, is malfunctioning. Chewbacca is working on the hyperdrive engine when he accidently crosses some wires that eject you from the Millennium Falcon while at the same time engaging the hyperdrive engine. Fortunately, you have a parachute inside of your clothing so you safely float onto the remote Calduum Island of Endor. Unfortunately, the Millennium Falcon is in hyperspace on the way to Hapsacean, and will return to Endor in 10 years. You have to survive on Calduum Island for 10 years. Calduum Island has industrious Ewoks and fertile farmland. If you are a wise leader of the Ewoks on Calduum Island, you can survive and joyously reunite with your good friends.

Calduum Island has exactly 100 Ewoks. The Ewoks are industrious, but they lack leadership. They immediately look to you as their leader. Calduum has fertile farmland and is the only climate on Endor that grows luscious gold smĭnts. Smĭnts is a grain similar to quinoa, which the Ewoks love to eat. The Ewoks are good farmers who own 1000 acres of fertile farmland and have 3000 bushels of luscious gold smĭnts in storage. You will lead Calduum Island, the Ewoks, farmland, and smĭnts to hopefully survive 10 years. If your leadership on Calduum Island is unwise, Ewoks will starve. If Calduum Island population decrease to 52 Ewoks, they will all swim away leaving your alone on Calduum, where you will be eaten by a womp rat never to see your friends again. If you make wise decisions when leading Calduum Island, Ewoks will survive for 10 years, at which time the Millennium Falcon will return from the Hapsacean Galaxy.

The luscious gold smĭnts can either be eaten or planted to grow more. The Calduum Island society requires smĭnts to plant, Ewok labor to sew and harvest smĭnts, farmland on which to plant smĭnts, and smĭnts for the Ewoks to eat. Smĭnts is planted and harvested on an annual basis. The following describe attributes of farming smĭnts and living on Calduum.

1. To plant 1 acre of land requires 2 bushels of smĭnts.
2. One Ewok can farm 10 acres of land.
3. Each Ewok requires 20 bushels of smĭnts per year to survive

The economy of Calduum uses a bushel of smĭnts as it currency. Calduum farmland is bought and sold by the acre. The price of farmland fluctuates annually. The value of farmland for the initial year is 20 bushels of smĭnts per acre.

You begin with 100 Ewoks, 1000 acres of fertile farmland, and 3000 bushels of smĭnts. You need to manage the amount of land you own, plant/harvest smĭnts on your land, and keep the Ewoks fed and alive. Each year you will have to make the following decisions as you lead Calduum Island.

1. Do you want to buy or sell land, and if so how many acres? Each year you can only do one operation – buy or sell – you cannot do both. Buying land consumes your smĭnts. Selling land adds to your smĭnts. You do not have to buy or sell land.
2. How many acres of land do you want to plant? Planting land consumes smĭnts and generates smĭnts, hopefully more than you planted.
3. How many bushels of smĭnts do you want to feed the Ewoks? The bushels of smĭnts for feeding will be a combination of smĭnts in storage, smĭnts used for planting, smĭnts gained from harvesting, smĭnts used for buying land, and smĭnts gained from selling land.

You must have enough smĭnts to conduct the operations resulting from your choices. The algorithm shall detect operations that cannot be supported. The sample console run in section 3.2 demonstrates an error message.

There are several acts of nature and economics that affect the outcomes.

1. Teeny-weeny-meteor shower – Teeny-weeny meteors occasionally crash on Calduum. Each year there is an 11% chance of a teeny-weeny-meteor shower. When this happens teeny-weeny meteors kill 28% of the Ewoks and 9% of the Ewoks swim away to another island.
2. Weather and harvest – For the most part Calduum has good weather, but it varies enough each year to affect the bushels per acre harvested. Weather is determined by generating random number between 1 and 9, inclusive. Each acre that was planted will yield this many bushels of

smĭnts. For example, if you planted 50 acres and the weather random number is 3, you harvest 150 bushels of smĭnts.

3. Womp rat – Occasionally a womp rat will swim to Calduum. Womp rats love luscious gold smĭnts just like Ewoks. The Ewoks slay the womp rat with their spears, slings, and knives, but not before the womp rat has eaten smĭnts. There is a 46% chance that a womp rat will swim to Calduum. When a womp rat swims to Calduum, it will eat between 1/25 and 4/10 of your smĭnts.
4. Land valuation – The price of land for the first year is 20 bushels per acre, and it changes each year. The price of land will be a random number between 17 and 26 bushels per acre (inclusive).
5. Immigrant Ewoks – Ewoks have a natural sense as to when Calduum smĭnts harvest is thriving, which causes Ewoks to swim to Calduum. If during a year, all of the Ewoks on Calduum are fed such that none of them die, new Ewoks will swim to Calduum according to the following formula. ((20 * number of acres + amount of smĭnts in storage) / (100 * population) + 1) * (random number between 0 and 1)

NOTE: all computations should result in integers, not floats.

## Endor Smĭnts – Sample Console Run

The following is a sample run of the Endor Smĭnts program. I have inserted explanatory comments in red. The explanatory comments are not part of the output of the program. Your output must be in the same order and format as this in order to make my testing easier.

The following prompts the user to be Princess Leia, Han Solo, or Luke Skywalker.

```
Select your Star Wars Character.
Princess Leia, Han Solo, Luke Skywalker: Princess Leia
```

The following is the introductory message explaining the game.

```
================================================================
========
You are Princess Leia, and you recently parachuted onto Calduu
m Island of the forest moon Endor. The Ewoks of Calduum Island
elected you to be their leader.  You have to keep Calduum Islan
d alive for 10 years, at which time the Millennium Falcon will
return and you will be reunited with your Star Wars friend.  Yo
u have to buy/sell land, direct smĭnts farming, and dispense sm
ĭnts to the Ewoks. Teeny-weeny-meteor showers and womp rats aff
ect your life on Calduum. The following will help you make deci
sions.
  * Each Ewok needs at least 20 bushels of smĭnts per year to s
urvive.
  * Each Ewok can farm at most 10 acres of land.
  * It takes 2 bushels of smĭnts to farm an acre of land.
  * The market price for land fluctuates yearly.
Initial values are the following.
  * Population: 100
  * Acres owned: 1000
  * Land Value: 20
  * Smints in Storage: 3000
Rule wisely for 10 years and you will be reunited with your fri
ends. Rule poorly, the Ewoks will die, you will perish, and yo
u will never see your friend again.
```

The following message provides the status of Calduum Island for a particular year, and prompts the user for decisions.

```
================================================================
========
Princess Leia, you are leading Calduum Island in the year 100
0.
The following events happened during the year 1000.
* Starved to death: 0 Ewoks
* Swam to Calduum: 0 Ewoks
* Swam away from Calduum: 0 Ewoks
* Teeny-weeny-meteors killed: 0 Ewoks.
* Womp rats destroyed: 0 bushels.
* Smĭnts harvested: 3000 bushels at 3 bushels per acre
At the end of the year 1000, Calduum is characterized by
* Population: 100 Ewoks
* Land Owned: 1000 acres
* Land value:  20 bushels per acre
* Smĭnts in storage: 3000 bushels
----------------------------------------------------------------
-------
How many acres do you wish to buy? 10
How many bushels of smĭnts do you wish to feed your Ewoks? 2000
How many acres do you wish to plant with smĭnts? 400
```

The following message provides the status of Calduum Island for a particular year, and prompts the user for decisions. In this iteration, the user tries to perform operations that are not supported with the smĭnts in storage.

```
================================================================
========
Princess Leia, you are leading Calduum Island in the year 100
1.
The following events happened during the year 1001.
* Starved to death: 0 Ewoks
* Swam to Calduum: 5 Ewoks
* Swam away from Calduum: 0 Ewoks
* Teeny-weeny-meteors killed: 0 Ewoks.
* Womp rats destroyed: 700 bushels.
* Smĭnts harvested: 1600 bushels at 4 bushels per acre
At the end of the year 1000, Calduum is characterized by
* Population: 105 Ewoks
* Land Owned: 1010 acres
* Land value:  21 bushels per acre
* Smĭnts in storage: 1500 bushels
----------------------------------------------------------------
-------
How many acres do you wish to buy? 0
How many acres do you wish to sell? 0
How many bushels of smĭnts do you wish to feed your Ewoks? 1200
How many acres do you wish to plant with smĭnts? 1000
================================================================
========
The following is an error message resulting from answers that r
equire more smĭnts than are in storage.  All error messages wil
l begin with ERROR: .

ERROR: Princess Leia, Think again, you do not have enough smĭnt
s in storage.
```

The following message provides the status of Calduum Island for a particular year, and prompts the user for decisions. These prompts continue until 10 years have passed or the Ewok population dips below 53.

```
================================================================
========
Princess Leia, you are leading Calduum Island in the year 100
1.
The following events happened during the year 1001.
* Starved to death: 0 Ewoks
* Swam to Calduum: 5 Ewoks
* Swam away from Calduum: 0 Ewoks
* Teeny-weeny-meteors killed: 0 Ewoks.
* Womp rats destroyed: 700 bushels.
* Smĭnts harvested: 1600 bushels at 4 bushels per acre
At the end of the year 1000, Calduum is characterized by
* Population: 105 Ewoks
* Land Owned: 1010 acres
* Land value:  21 bushels per acre
* Smĭnts in storage: 1500 bushels
----------------------------------------------------------------
-------
How many acres do you wish to buy? 0
How many acres do you wish to sell? 0
How many bushels of smĭnts do you wish to feed your Ewoks? 1200
How many acres do you wish to plant with smĭnts? 150
================================================================
========
Princess Leia, you are leading Calduum Island in the year 100
2.
The following events happened during the year 1002.
* Starved to death: 40 Ewoks
* Swam to Calduum: 0 Ewoks
* Swam away from Calduum: 0 Ewoks
* Teeny-weeny-meteors killed: 0 Ewoks.
* Womp rats destroyed: 0 bushels.
* Smĭnts harvested: 750 bushels at 5 bushels per acre
At the end of the year 1000, Calduum is characterized by
* Population: 65 Ewoks
* Land Owned: 1010 acres
* Land value:  22 bushels per acre
* Smĭnts in storage: 900 bushels
...
```

## Endor Smĭnts – Individual, Team

For the Endor Smĭnts Project, you shall complete the following.

1. Read the Endor Smĭnts Project description to make sure you understand the requirements. This is a team component.
2. Create collection of test cases that you will use to test your Endor Smĭnts Project. This is a team component. Further guidance on developing the test cases in provided in Test Case Guidance Section.
3. Create a design of your Endor Smĭnts Projects. This is a team component and an individual component. As a team you can discuss a general design, but each student has to create their own detailed design. Further guidance on developing the design is provided in Design Guidance Section.
4. Write a program that allows a user to interact via the console to play the game as describe in the Description and Requirements sections. This is an individual component. Further guidance on developing the program is given in Programming Guidance Section. Your program must be properly documented. If your program includes Java classes, you must include JavaDoc explaining your classes. Your program must present information to the user as described in the previous Section. You must adhere to the wording and order. The Design Guidance Section and the Programming Guidance Section provide helpful hints on development. The Programming Guidance Section leads you through one way of developing the code. If you follow this style, you can use my EndorGui.java to run your program in a GUI. You do not have to adhere to this design or implementation. You must adhere to the user interaction described in show in the previous Section.

There are two due dates for the Endor Smĭnts project, which are as specified in Canvas.

1. First Due Date – The Endor Smĭnts test cases and design are due. These will be submitted as printed documents at the start of class. Each team should submit their test cases. Each student should submit his or her design.
2. Final Due Date – The Endor Smĭnts program is due. Each student will submit the .java files. You must submit all files necessary to run your Endor Smĭnts program.

## Endor Smĭnts – Test Case Guidance

You can use the Endor Smĭnts requirements and the sample execution in from the Description and Requirements section to create test cases before creating code. Many software organizations have a testing group that is solely responsible for testing. The programmers and the testers both use the requirements to develop

their products. Your team should create test cases that will sufficiently test your Endor Smĭnts code. Your thinking should include error cases and good cases. The following are a few example test cases.

```
Case 1: Feeding Ewoks and buying farmland consumes more smĭnts
than available
Buy: 60
Feed: 2000                2000 + 60*20 yields 3200 bushels, err
or – cannot do
Plant: 0

Case 2: Lose on first turn
Buy: 1
Feed: 1000                1000 will feed 50 Ewoks, so populatio
n < 53, game over
Plant: 100

Case 3: Use all smĭnts on turn one, and then sell farmland on t
urn two to feed Ewoks
Step 1: Buy: 0, Sell: 0
Feed: 3000                Overfeed Ewoks to make smĭnts in stor
age equal 0
Plant: 0               We may attract some immigrant Ewoks

Step 2: Buy: 0
Sell: 500                Selling 500 will yield at least 500*1
7 or 8500 bushels to feed
Feed: 2000                Ewoks will not die unless teeny-ween
y-meteor shower occurs
Plant: 0
```

## Endor Smĭnts - Design Guidance

The Endor Smĭnts Calduum Island game is a simulation. Input, the simulation attributes, the simulation algorithm that changes the simulation attributes, and output represent a simulation. This view of a simulation is captured in the following diagram.

A simulation will execute the preceding diagram in a loop. On each loop iteration the attributes describe a specific state of the simulation.

For the Endor Smĭnts Calduum Island simulation, the attributes, inputs, and outputs as follows.

### Attributes

The attributes describe the current state of your simulation. They are persistent. The must be stored in a variables that persist across each execution of the algorithm.

- The leader (e.g. Princess Leia)
- The population of Ewoks
- The number of acres of farmland
- The number of bushels of smĭnts in storage.

### Inputs

- The number of acres of land to buy or sell
- The number of acres of land to plant
- The number of bushels of Smĭnts to feed to the Ewoks.

### Outputs

- The number of Ewoks starved to death.
- The number of Ewoks swam to Calduum.
- The number of Ewoks swam away from Calduum,
- The number of Ewoks killed by teeny-weeny-meteors,
- The number of bushels of smĭnts womp rats destroyed
- The number of bushels harvested.

## Endor Smĭnts – Programming Guidance

This section describes one way to write the program. There are many ways to write this program. You are free to write the program that adheres to your design. This program structure consists of two components – (1) a class that can be used to construct a Calduum Island simulation object and (2) a main program class that constructs a Calduum Island simulation object and interacts with the user to display/collect information. The Calduum Island simulation class has a method that simulates one year on Calduum Island. This section names the Calduum Island simulation class EndorSmints. An alternative name would be something like CalduumSimulation class.

### Instance Variables

All of the attributes described in Design Guidance Section are instance variables.

### Constructors

`public EndorSmints(int year, int population, int acresOfLand, int landValue, int smintsI` – Allows the user to construct an EndorSmints object with selected values. Note the leader parameter will be “Princess Leia”, “Han Solo”, or “Luke Skywalker”.

`public EndorSmits()` – calls the previous constructor with the initial values defined in Description and Requirements Section.

### Methods

- `public String introMessage()` – computes the introductory message, which is shown in Description and Requirements Section.
- `public String simulateYear(int acresToBuy, int acresToSell, int acresToPlant, int` – Simulates one year on Calduum Island using an algorithm that satisfies the requirements specified in Description and Requirements Section. This method returns a String that provides the status of Calduum Island at the end of the simulated year. An example of this String is shown in Description and Requirements Section.

Depending upon the design of your program, you may need to create getter/setter and various helper methods such as the following.

```
public int getPopulation()
public int getAcres()
```

### Program Benefits

This style of program separates the user interaction and the simulation. You can use the simulation with console-based I/O (as shown in this specification) or with GUI-based I/O.

### Stepwise Approach to Creating Code

When creating a complex class such as EndorSmits, it is beneficial to take a stepwise approach. This means you create a little bit of the code, test it, add some code, test it, and continue until you have completed the entire program. The following is an example of stepwise development.

- Step 1a: Create the class EndorSmits, the constructors, and a few getter/setter methods. You will have the following instance variables.

```
private int year;
private int population;
private int acresOfLand;
private int landValue;  // in bushels per acre
private int smitsInStorage;
```

- Step 1b: Write a separate class for your main program that will create an EndorSmits object and interact with the user. Use this main program test your skeleton EndorSmits class.
- Step 2a: Add the method introMessage() to your EndorSmits class. The returned String should use your instance variables to create the String.
- Step 2b: Update your main program class to test your introMessage() method.
- Step 3a: Add a capability described in the Description and Requirements section to your EndorSmits class.
- Step 3b: Update your main program class to test the capability you just added. This will involve playing a game to see if what you implemented is correct. You will have to think hard to determine if you have implemented the capability correctly. You will have to play a year, examine the results, maybe compute the correct answer by hand, and compare to your program's results.

You will iterate on Steps 3a and 3b until you have completed implementing the method simulateYear(). Step 3a adds capabilities to simulateYear() and step 3b tests what you have added. You should study the Description and Requirements section to determine the order you want to add capabilities. For example, on your first iteration of step 3, you may implement the teeny-weeny-meteor shower attribute. After adding the teeny-weeny-meteor shower, you may add the harvest attribute.

- Step 4: After you have implemented console-based I/O program, you may want to implement a GUI-base I/O program. If you have simulation class that is not dependent upon a particular I/O, replacing the console-base I/O program with a GUI-based I/O program is not too difficult.

# Endor Smĭnts – Simulation Enhancements

The following are ways that the Smĭnts simulation could be enhanced. These are not requirements; however, if you implement an enhancement, I will award you extra points. Do not attempt enhancements until you have completed the basic Endor Smĭnts program.

1. Update the teeny-weeny-meteor shower event such that a random amount between 25-50% of the Ewoks die and a random number between 0-10% leave.
2. Update the amount of land an Ewok can farm. Currently each Ewok can plant 10 acres each. If you have a population of 80, you can plant 800 acres of maize. Update the event such that in a year immediately after a teeny-weeny-meteor shower, Ewoks can only plant 7 acres each.
3. Update the termination condition so that if the ratio of Ewoks to acres of farmland drops below 1/20, all of the Ewoks swim away and the game terminates.
4. Before the game commences ask the user if he wants to play at easy, standard, or challenge level. For example, perhaps the easy level has no womp rats, or teeny-weeny-meteor showers, or Ewoks eat less. The challenging level might mean Ewoks eat more smĭnts and the probability of a teeny-weeny-meteor shower is greater.
5. Add a feature of your own.

# Battleship

**Learning Objective(s):**

- Understand Java interfaces.
- Understand Java packages.
- Understand how to retain state between calls of methods.
- Understand how to use char[] for implementing game boards.

## Battleship

Stephen Davies created this project. I edited the description.

For the Battleship Project, you will write a program that plays the game Battleship. Your program will implement a gaming algorithm that maintains a state, uses 2-dimensional arrays, and double dispatch. Your program can compete in live game play against your fellow students to see who implements the best Battleship algorithm.

For this project you will not be submitting a main() method. You will develop three methods that are called by a main() method that plays students Battleship games against each other.

## Battleship – The Game

If you have never played Battleship, you should read the Wikipedia Battleship article (https://en.wikipedia.org/wiki/Battleship_(game)) to get informed.

Battleship is a popular game in which two players hide a fleet of ships in a sea and try to sink the others before their own go down. The standard game is played on a 10x10 board. Each player hides five ships on the squares of this board. The five ships and their lengths are as follows:

- Aircraft carrier - 5 squares long
- Battleship - 4 squares long
- Submarine - 3 squares long
- Destroyer - 3 squares long
- Patrol boat - 2 squares long

Each ship must be placed vertically or horizontally (not diagonally) on the board. Two ships can occupy adjacent squares, but ships cannot overlay each other.

Players take turns shooting violently at one another. On a player's turn, he or she calls out the row and column of a square on the other person's board that he or she wants to shoot at. An example shot is "row 4, column 3!" The opponent replies by telling the shooter which ship, if any, is at that location. An example reply is "you hit my battleship." The shooter dutifully records this information for future use, and then responds to their opponent when they similarly call a square.

The first player to completely sink all 5 of their opponent's ships wins the game. Note that "sink" doesn't mean "hit just once" but rather "hit every square of the ship." In other words, to sink an aircraft carrier, the player must, on five of their turns, call out each of the 5 squares that the aircraft carrier occupies.

Normally, players play only two or three games before they get bored, but we're going to have a simulator playing thousands of games, with the overall winner claiming the prize.

Note that it is possible to tie in a game of Battleship, if the two players each hit their opponent's last ship square on the same (last) move.

## Battleship – Individual, Team, and Due Dates

For the Battleship Project, you will complete the following.

1. Write a jsmith_BattleshipPlayer class that implements the BattleshipPlayer interface. This is an individual component. The details for writing jsmith_BattleshipPlayer are provided in the next section. The jsmith_BattleshipPlayer implements three methods.
    - a. A method that hides your fleet of ships on an empty grid
    - b. A method that shoot at the opponent when called to do so.
    - c. A method that resets your object at the start of a new game.
2. Test your jsmith_BattleshipPlayer against yourself and some of your team members. The Testing Section provides guidance on how to create a tester. This is a team component.

Your Java class for Battleship will be stateful. This means that your Battleship class will declare instance variables that retain the state of your game. This allows you to have memory between moves. You must remember what you've shot at, and what was revealed on each square, so that you can intelligently decide where to shoot next. The details of this are explained in the next section.

The due date for submitting your Battleship Project is as specified on Canvas.

# Battleship – Implementation Details

In order to create a Battleship program that plays correctly, you have to follow the implementation details described in this section.

Your Java class for Battleship will be stateful. This means that your Battleship class will declare instance variables that retain the state of your game. This allows you to have memory between moves. You must remember what you've shot at, and what was revealed on each square, so that you can intelligently decide where to shoot next.

## Files Provided to Create Your Class

The following files are provided to you, which will be used as you edit your project in Netbeans.

- Board.java
- BattleshipPlayer.java
- jsmith_BattleshipPlayer.java

## Your Battleship Project in Netbeans

To establish your project in Netbeans, do the following:

1. Create a new project. Call it "Battleship." (Please do not call it anything else. Please call it Battleship, capitalized.)
    - a. Ensure the Create Main Class is checked.
    - b. In the text field next to the Create Main Class, enter battleship.BattleshipTester
    - c. This creates a package, battleship, in your project, and it adds the BattleshipTester.java file to the package. For now, you can ignore editing BattleShipTester.
2. This step adds the Board.java file to your battleship package. There are two ways to do this.
    - a. Copying the contents from Board.java into a file while using the Netbeans editor.
        - i. Right-click on the battleship (lower-case) package that Netbeans created inside your Battleship (capitalized) project, and choose "New Java Class."

        - ii. Name the new class Board.
        - iii. Open up the Board.java file you just created. Delete its entire contents, and replace it with the contents of the Board.java file provided to you.
    - b. Copying the file Board.java into the folder underlying the Netbeans package.
        - i. Use your Explorer/Finder to copy the Board.java into the Netbeans folder for the battleship package. On my system, the target folder is

```
.../Netbeans/Battleship/src/battleship
```

3. This step adds the battleship.players package to your project.
    - a. Right-click on the battleship (lower-case) package that Netbeans created inside your Battleship (capitalized) project, and choose “New Java Package.”
    - b. Name the new package battleship.players, all lower-case, with no spaces, and with a dot between the two p’s, just as shown.
4. This step adds the BattleshipPlayer.java file to your battleship.players package. There are two ways to do this.
    - a. Copying the contents from Battelship.java into a file while using the Netbeans editor.
        - i. Right-click on the battleship.players package that you just created, and choose “New Java Interface.”
        - ii. Name the new interface “BattleshipPlayer.”
        - iii. Open up the BattleshipPlayer.java file you just created. Delete its entire contents, and replace it with the contents of the BattleshipPlayer.java file provided to you.
    - b. Copying the file Board.java into the folder underlying the Netbeans package.
        - i. Use your Explorer/Finder to copy the Board.java into the Netbeans folder for the battleship/players package. On my system, the target folder is

```
.../Netbeans/Battleship/src/battleship/pla
yers
```

5. This step adds the jsmith_BattleshipPlayer.java file to your battleship.players package, where jsmith is your UMW userid.. There are two ways to do this, but using the Netbeans editor is probably easiest for most.
    - a. Copying the contents from Battelship.java into a file while using the Netbeans editor.
        - i. Right-click on the “battleship.players” package that you created, and choose “New Java Class.”
        - ii. Name the new class jsmith_BattleshipPlayer, where jsmith is your UMW userid. Please do not name the class literally jsmith_BattleshipPlayer. Rather, name it (for example) rjones2_BattleshipPlayer, if rjones2 is your userid. Please do not name the class anything else. Please do not get clever, or creative, or capitalize differently, or use a hyphen instead of an underscore, or deviate from this naming convention in any way.
        - iii. Open up the rjones2_BattleshipPlayer.java (or whatever) file you just created. Delete its entire contents, and replace it with the contents of the jsmith_BattleshipPlayer.java file provided to you. iv. Note that when you do this, you will immediately have an error because the class name does not match the file name. You should fix this error by changing jsmith to your userid in the “public class” line.
    - b. This step assumes your UMW userid is rjones2. Rename the file jsmith_BattleshipPlayer.java to rjones2_BattleshipPlayer.java. Copy the rjones2_BattleshipPlayer.java into the Netbeans folder for the battleship/players package. On my system, the target folder is

    ```
    .../Netbeans/Battleship/src/battleship/players
    ```

6. You are now ready to begin your mission of implementing the hideShips(), go(), and reset() methods.

## Packages

As can be determined from the above instructions, this project consists of two packages: one called battleship, and the other called battleship.players. This is an extremely common practice in object-oriented programming, and is akin to organizing files within folders. The Board class (as well as the simulator class, and your main method) will be in the battleship package. The BattleshipPlayer.java interface, as well as your own (and all your fellow students') implementations of the player, will be in the battleship.players package.

## Your methods

Your rjones2_BattleshipPlayer.java file has detailed comments describing the hideShips(), go(), and reset() methods you are to write code for. Reading these detailed comments is an excellent and praiseworthy idea.

The method hideShips() returns a new Board object that has your ships hidden within it. Read the comments in the Board class for information on how to use its constructor. It takes a 2-dimensional array of characters as an argument, representing your board. Ships are represented in this array with the character of their first letter (capital 'A' for aircraft carrier, capital 'B' for battleship, etc.) and all empty squares are represented with the space character ' '. Recall that in Java, a character is delimited by single quotation marks, unlike Strings, which use double quotes.

A very important note about the hideShips() method: it must not return the same board twice in a row. This is partially for your own good (if you hide your ships in the same place each time, you're mincemeat for clever programmers who will remember where you put them and literally blow you out of the water in 17 moves the next round) and partially because I want you to think about how to ensure that this happens. Hint: use an instance variable.

The go() method is where you'll spend most of your time. It uses a technique called "double dispatch" in which it is given an argument that it calls method on. The argument is a Board object, and your job in go() is to choose a square to fire at and call the fireAt() method once to fire at that square. It will return the contents of that square, which you'll probably want to remember for future use. Note that calling fireAt() more than once per turn is not only cheating, it's detectable cheating, and my program will catch you and flog you if you try to do it.

The reset() is called when the game is over and a new game is starting. The control program will play a 10,000 game sequence. The reset() method gives your object a chance to catch its breath and prepare itself for a new game.

# Battleship Tournament

Your rjones2_BattleshipPlayer.java file is due on April 7. I will collect all of the .java files from Canvas, and we will have a Battleship Tournament in class on April 12. We will hold a world cup bracket-style Battleship tournament in which all functioning programs are entered as participants. Drama and excitement will be aplenty as the big screen shows the action. The grand prizewinner will receive a valuable prize.

# Grading

You have two grading options for this assignment. You can go for a B, or you can go for an A. It is easy to get a B on the Battleship Project; however, it is difficult to get an A.

### For a B…

To receive a B for this assignment, it is only necessary that your methods return a correct answer. In other words, your hideShips() method must always return a valid Board, and never the same one twice in a row. Your go() method must always fire at a valid square. And your reset() method must not barf. Beyond that, it doesn't matter how "dumb" your methods are, or how badly they play: they only must play according to the rules.

Note that in order to confirm that your methods are returning correct answers, you will need to thoroughly test your code by giving it a variety of inputs. This is part of your challenge: what are the different cases? (See notes on "Testing," below.)

### For an A…

To get an A, you must go beyond simply legality and attempt to implement an intelligent strategy for playing the game. You must strive to fire not only at a legal square, but a good square. Think about how you actually play the game of Battleship. For example questions consider the following. Where do you hide your ships? How do you choose where to fire next? Try to write code that imitates your thought process.

The criteria for judging whether you attempted to implement an intelligent strategy is given by the following.

1. If you are one of the final two players in the Battleship tournament, your program will automatically and unquestionably judged to be of A quality.

2. If you do not make it to the final two players, I will consider how far you advanced. I will also examine your compellingly documented code to judge whether you attempted a non-trivial, intelligent approach. Your comments should narrate your algorithm completely and transparently. They should shed light on what your code is doing, and why, and explain the theory behind your approach. I will award "A-range" points solely at my discretion based on how thorough and creative your program looks to me.

For something worse…

Note that your program cannot be entered into the tournament if it does not meet the "for a B…" criteria above. This is for the simple reason that it will crash my program. Only programs that return correct answers will be eligible for the tournament, which will be tested prior to our tournament.

# Testing

This section describes how to begin testing your project. As you gain experience with Battleship, we can discuss a GUI Tester developed by previous students.

To test your Battleship player, you're going to have to write some code in main method. You initial tester will probably be short. The strategy for your tester is to create two BattleshipPlayers using your BattleshipPlayer, and then play them against each other. The following shows how to create two BattleshipPlayers that use my userid.

```
ecooper_BattleshipPlayer a = new ecooper_BattleshipPlayer();
ecooper_BattleshipPlayer b = new ecooper_BattleshipPlayer();
```

After creating your players, you can create a loop where (on each loop iteration) they fire at each other. Of course your two Battleship players will have to hide a Board prior to playing, which will be what is fired at – a will fire at b's Board and b will fire at a's Board.

The Board class has a field and a method that are useful in your tester.

- firedAtThisRound – you have to set this field to false on each iteration of the loop
- isComplete – this method returns true when a board is complete, meaning all of the ships are sunk.

The following is an example tester.

```
package battleship;

import battleship.players.ecooper_BattleshipPlayer;

/**
 * Created by yoda
 */
public class BattleshipTester {
    public static void main(String[] args) {
        ecooper_BattleshipPlayer a = new ecooper_BattleshipPlay
er();
        ecooper_BattleshipPlayer b = new ecooper_BattleshipPlay
er();

        Board aBoard = a.hideShips();
        System.out.println(aBoard);
        Board bBoard = b.hideShips();
        System.out.println(bBoard);
        a.reset();
        b.reset();

        int step = 1;

        for(int i = 0; i < 9; i++) {
            System.out.println("STEP " + step + "
================");
            bBoard.firedAtThisRound = false;
            a.go(bBoard);
            System.out.println(bBoard);
            aBoard.firedAtThisRound = false;
            b.go(aBoard);
            System.out.println(aBoard);
            if (aBoard.isComplete()) {
                System.out.println("B Wins!");
                break;
            }
            if (bBoard.isComplete()) {
                System.out.println("A Wins!");
                break;
            }
            step++;
        }
    }
}
```

# Your Choice

**Learning Objective(s):** Your Choice

## Choice

Project 5 is your choice.